4.Objectives

- Explain how to resolve the conflict during merge.

In this hands-on lab, you will learn how to:

- Implement conflict resolution when multiple users are updating the trunk (or master) in such a way that it results into a conflict with the branch's modification.

Prerequisites

The following are the pre-requisites to complete this hands-on lab:

- Hands-on ID: **"Git-T03-HOL_001"**

Notes*:

Please follow the below steps for creating a free account in GitHub. Do not use cognizant credentials to login to GitHub.

Estimated time to complete this lab: **30 minutes.**

Please follow the instructions to complete the hands-on. Each instruction expect a command for the Git Bash.

1. Verify if master is in clean state.
2. Create a branch **"GitWork".** Add a file "hello.xml".
3. Update the content of "hello.xml" and observe the status
4. Commit the changes to reflect in the branch
5. Switch to master.
6. Add a file **"hello.xml"** to the master and add some different content than previous.
7. Commit the changes to the master
8. Observe the log by executing **"git log –oneline –graph –decorate –all"**
9. Check the differences with Git diff tool
10. For better visualization, use P4Merge tool to list out all the differences between master and branch
11. Merge the bran to the master
12. Observe the git mark up.

13. Use 3-way merge tool to resolve the conflict
14. Commit the changes to the master, once done with conflict
15. Observe the git status and add backup file to the .gitignore file.
16. Commit the changes to the .gitignore
17. List out all the available branches
18. Delete the branch, which merge to master.
19. Observe the log by executing **"git log –oneline –graph –decorate"**

Answer:

What is a Merge Conflict?

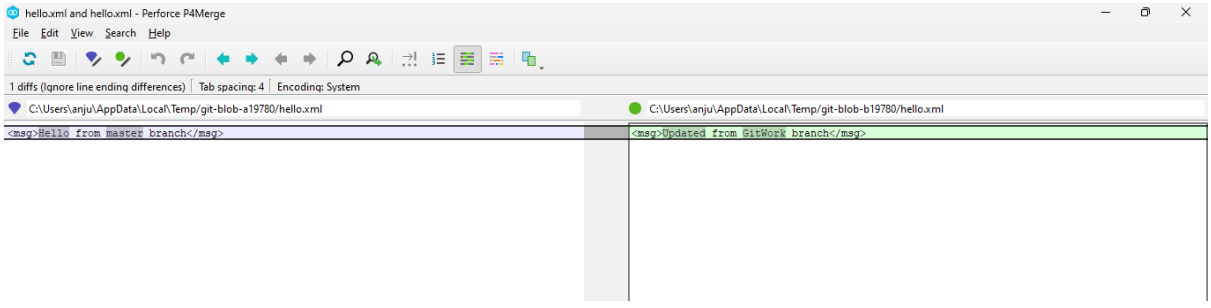
A **merge conflict** happens when Git is unable to automatically combine changes from two branches. This usually occurs when two people (or branches) change the same part of a file in different ways.

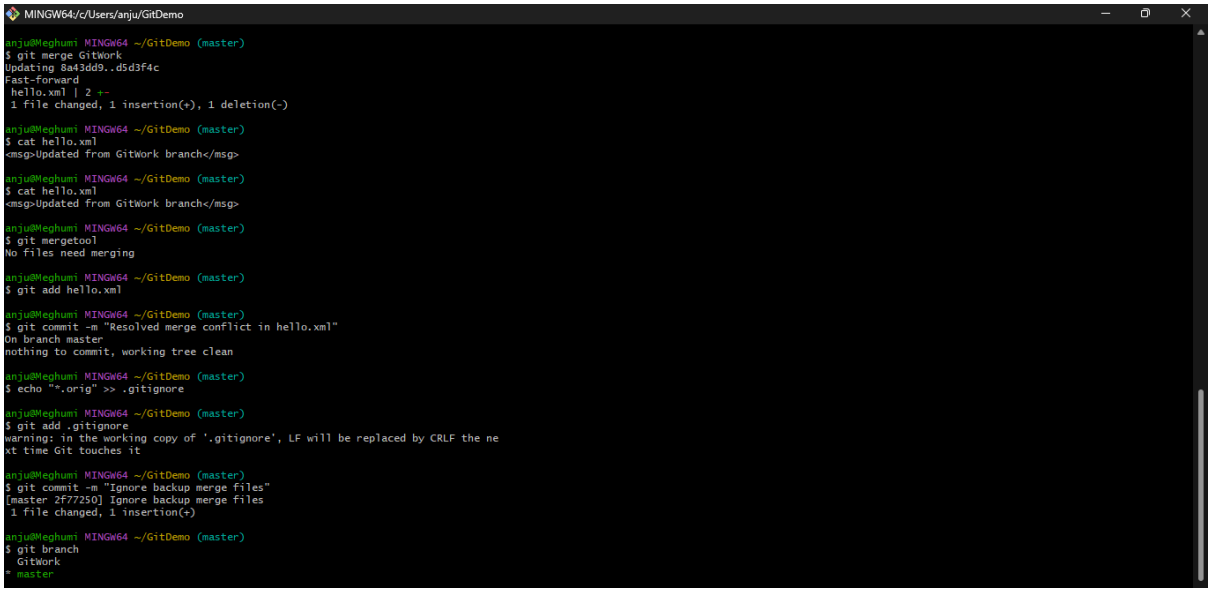
How to Resolve a Merge Conflict :

1. **Start the merge process**: You attempt to combine two branches. Git will try to merge them automatically.
2. **Conflict is detected**: If Git cannot decide which change to keep, it marks the file as conflicted and pauses the merge.
3. **Identify the conflicting files**: You check which files have conflicts. Git clearly shows you which files need attention.
4. **Open the file with conflict**: Inside the file, Git marks the conflicting sections using special markers. These markers show both versions of the conflicting code.
5. **Manually resolve the conflict**: You read the conflicting changes and decide what the final version should be. You can choose one version, or combine both.
6. **Remove the conflict markers**: After choosing the correct version, you remove Git's conflict indicators from the file to clean it up.
7. **Mark the conflict as resolved**: Once the conflict is resolved, you let Git know that you have handled the problem.
8. **Complete the merge process**: Finally, you save your changes and complete the merge. The branches are now successfully merged.

```
anju@Meghumi MINGW64 ~
$ cd GitDemo

anju@Meghumi MINGW64 ~/GitDemo (master)
$ git status
On branch master
nothing to commit, working tree clean

anju@Meghumi MINGW64 ~/GitDemo (master)
$ git checkout master
Already on 'master'

anju@Meghumi MINGW64 ~/GitDemo (master)
$ git status
On branch master
nothing to commit, working tree clean

anju@Meghumi MINGW64 ~/GitDemo (master)
$ git checkout -b GitWork
Switched to a new branch 'GitWork'

anju@Meghumi MINGW64 ~/GitDemo (GitWork)
$ echo "<msg>Hello from GitWork branch</msg>" > hello.xml

anju@Meghumi MINGW64 ~/GitDemo (GitWork)
$ echo "<msg>Updated from GitWork branch</msg>" > hello.xml

anju@Meghumi MINGW64 ~/GitDemo (GitWork)
$ git status
On branch GitWork
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   hello.xml

no changes added to commit (use "git add" and/or "git commit -a")

anju@Meghumi MINGW64 ~/GitDemo (GitWork)
$ git add hello.xml
warning: in the working copy of 'hello.xml', LF will be replaced by CRLF the nex
t time Git touches it

anju@Meghumi MINGW64 ~/GitDemo (GitWork)
$ git commit -m "Added hello.xml in GitWork branch"
[GitWork d5d3f4c] Added hello.xml in GitWork branch
 1 file changed, 1 insertion(+), 1 deletion(-)
```

```
anju@Meghumi MINGW64 ~/GitDemo (GitWork)
$ git checkout master
Switched to branch 'master'

anju@Meghumi MINGW64 ~/GitDemo (master)
$ echo "<msg>Hello from master branch</msg>" > hello.xml

anju@Meghumi MINGW64 ~/GitDemo (master)
$ git add hello.xml
warning: in the working copy of 'hello.xml', LF will be replaced by CRLF the nex
t time Git touches it

anju@Meghumi MINGW64 ~/GitDemo (master)
$ git commit -m "Added hello.xml in master branch"
On branch master
nothing to commit, working tree clean

anju@Meghumi MINGW64 ~/GitDemo (master)
$ git log --oneline --graph --decorate --all
* d5d3f4c (GitWork) Added hello.xml in GitWork branch
* 8a43dd9 (HEAD -> master) Add backup files to .gitignore
* 965fac0 Master branch: added hello.xml
* de58e59 GitWork branch: updated hello.xml

anju@Meghumi MINGW64 ~/GitDemo (master)
$ git difftool master GitWork

Viewing (1/1): 'hello.xml'
Launch 'p4merge' [Y/n]? Y

anju@Meghumi MINGW64 ~/GitDemo (master)
$ git config --global merge.tool p4merge

anju@Meghumi MINGW64 ~/GitDemo (master)
$ git config --global mergetool.p4merge.cmd '"C:/Program Files/Perforce/p4merge.
exe" "$BASE" "$LOCAL" "$REMOTE" "$MERGED"'

anju@Meghumi MINGW64 ~/GitDemo (master)
$ git difftool master GitWork

Viewing (1/1): 'hello.xml'
Launch 'p4merge' [Y/n]? Y
```




MINGW64:/c/Users/anju/GitDemo
anju@Meghumi MINGW64 ~/GitDemo (master)
$ git merge GitWork
Updating 8a43dd9..d5d3f4c
Fast-forward
hello.xml | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
anju@Meghumi MINGW64 ~/GitDemo (master)
$ cat hello.xml
<msg>Updated from GitWork branch</msg>
anju@Meghumi MINGW64 ~/GitDemo (master)
$ cat hello.xml
<msg>Updated from GitWork branch</msg>
anju@Meghumi MINGW64 ~/GitDemo (master)
$ git mergetool
No files need merging
anju@Meghumi MINGW64 ~/GitDemo (master)
$ git add hello.xml
anju@Meghumi MINGW64 ~/GitDemo (master)
$ git commit -m "Resolved merge conflict in hello.xml"
On branch master
nothing to commit, working tree clean
anju@Meghumi MINGW64 ~/GitDemo (master)
$ echo "*.orig" >> .gitignore
anju@Meghumi MINGW64 ~/GitDemo (master)
$ git add .gitignore
warning: in the working copy of '.gitignore', LF will be replaced by CRLF the ne
xt time Git touches it
anju@Meghumi MINGW64 ~/GitDemo (master)
$ git commit -m "Ignore backup merge files"
[master 2f77250] Ignore backup merge files
1 file changed, 1 insertion(+)
anju@Meghumi MINGW64 ~/GitDemo (master)
$ git branch
GitWork
* master


anju@Meghumi MINGW64 ~/GitDemo (master)
$ git branch
GitWork
* master
anju@Meghumi MINGW64 ~/GitDemo (master)
$ git branch -d GitWork
Deleted branch GitWork (was d5d3f4c).
anju@Meghumi MINGW64 ~/GitDemo (master)
$ git log --oneline --graph --decorate
* 2f77250 (HEAD -> master) Ignore backup merge files
* d5d3f4c Added hello.xml in GitWork branch
* 8a43dd9 Add backup files to .gitignore
* 965fac0 Master branch: added hello.xml
* de58e59 GitWork branch: updated hello.xml